AI-Enabled IoT-Based Smart Irrigation and Crop Monitoring System Using Java

Initial Software Project Proposal

Student Name:Md Mahmudur Rahman
Student ID:IT-24024
Department : Information and Communication Technology

Course: Software Engineering
Submission Date: _____

Contents

1 Executive Summary **1**

2 Introduction **2**

2.1 Problem Statement 2

2.2 Aim 2

2.3 Objectives 2

2.4 Scope 3

2.4.1 Included in the initial version 3

2.4.2 Not included in the initial version 3

2.5 Stakeholders and Users 3

3 Requirements Specification **5**

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 6

3.3 Assumptions and Constraints 6

4 Feasibility and Risk Analysis **8**

4.1 Feasibility 8

4.2 Risk Register 8

5 Software Process Model **10**

5.1 Iterations 10

6 Project Schedule, Milestones, and Deliverables **11**

6.1 Eight-Week Schedule 11

6.2 Gantt Chart 12

7 System Architecture and AI Plan **13**

7.1 Architecture 13

7.2 AI Problem Definition 13
7.3 Training Workflow 13
7.4 AI Limitations 14

8 System Diagrams 15
8.1 Data Flow Diagram (Level 1) 16
8.2 Use Case Diagram 17
8.3 Activity Diagram 18
8.4 Entity Relationship Diagram 19
8.5 Class Diagram 20
8.6 Object Diagram 21

9 Implementation Plan and Interface Prototype 22
9.1 Suggested Software Structure 22
9.2 Java Class Responsibilities 22
9.3 Prototype Features 22
9.4 Important AI Implementation Note 23

10 Testing and Evaluation 24
10.1 Test Plan 24
10.2 AI Evaluation Metrics 25

11 Expected Outcomes 26

12 Future Enhancements 27

13 Conclusion 28

14 References 29

A Submission Notes 30

List of Figures

6.1 Planned eight-week project schedule. 12

8.1 Level 1 data flow: measurements, validation, inference and review. 16
8.2 Use cases within the system boundary; lines denote actor associations. . . 17
8.3 Activity workflow with invalid-reading rejection and independent safety checks. 18
8.4 Logical entity relationships; PK = primary key and FK = foreign key. . . 19
8.5 Simplified UML class relationships; hollow triangle indicates inheritance. 20
8.6 Illustrative object instances; sample values are not field-validated. 21

Chapter 1

Executive Summary

The proposed project, *AI-Enabled IoT-Based Smart Irrigation and Crop Monitoring System Using Java*, is an academic prototype that monitors soil and environmental conditions and provides irrigation recommendations. An ESP32 reads sensor values, such as soil moisture, temperature, and humidity, and sends them to a Java-based application. The application stores readings, displays current and historical conditions, and requests a machine-learning model to estimate whether irrigation may be needed. The first version will operate in monitoring and recommendation mode. Automatic pump control will only be enabled after hardware checks, model evaluation, and independent safety controls have been implemented.

The planned stack includes ESP32, a capacitive soil-moisture sensor, a temperature/humidity sensor, MQTT, Java Spring Boot, MySQL, a Python/scikit-learn model or compatible Java inference library, and a browser-based dashboard. A small Java desktop prototype is also supplied to demonstrate class-based interface design. The project follows an iterative software process and is planned for eight weeks.

Chapter 2

Introduction

Agriculture depends on appropriate water management. Irrigation that is too early or too frequent can waste water, while delayed irrigation can stress crops. A monitoring system can provide timely measurements and make the decision process more consistent. This project combines Internet of Things (IoT) sensing, machine learning, and Java application development in one demonstrable system.

2.1 Problem Statement

Small-scale growers may not have continuous access to soil-condition measurements or a simple way to review historical readings. A fixed moisture threshold can be useful but may not reflect crop type, recent rainfall, or changing environmental conditions. The proposed system will collect sensor readings and provide data-based recommendations. The model's performance will be compared with a simple threshold baseline so that the project does not assume AI is automatically better.

2.2 Aim

To design and implement a prototype that collects field measurements, stores them, displays them, and uses an evaluated machine-learning classifier to recommend whether irrigation may be needed.

2.3 Objectives

1. Read soil moisture, air temperature, and relative humidity using an ESP32 and suitable sensors.
2. Transmit readings to the application through MQTT or HTTP.
3. Store timestamped readings and prediction results in MySQL.

4. Train and evaluate a supervised-learning classification model using appropriately labeled data.
5. Integrate model inference with the Java backend.
6. Provide a dashboard showing current readings, predictions, device status, and history.
7. Add manual pump control and independent safety limits; keep automatic operation disabled by default.
8. Produce system diagrams, test evidence, a Gantt chart, and a professional report.

2.4 Scope

2.4.1 Included in the initial version

- Device registration and sensor-data ingestion.
- Soil-moisture, temperature, and humidity display.
- Historical reading storage and simple graphs.
- AI-assisted binary classification: “irrigation may be needed” or “irrigation may not be needed”.
- Prediction history and model-version tracking.
- Manual recommendation review and basic alert display.
- A Java interface prototype demonstrating class relationships.

2.4.2 Not included in the initial version

Online marketplace functions, commercial-scale deployment, guaranteed crop-yield prediction, medical or safety-critical use, autonomous operation without supervision, and claims of water savings without controlled evaluation are outside the initial scope.

2.5 Stakeholders and Users

Stakeholder	**Interest / responsibility**
Farmer / grower	Reviews measurements and irrigation recommendations; may confirm or reject a recommendation.

Administrator	Registers devices, manages accounts, reviews system status, and checks logs.
Student developer	Designs, implements, tests, documents, and demonstrates the prototype.
Course instructor	Evaluates project scope, software process, diagrams, implementation, and report.

Chapter 3

Requirements Specification

3.1 Functional Requirements

ID	Requirement
FR-01	The system shall accept sensor readings from a registered device.
FR-02	The system shall validate readings and record their timestamp and device ID.
FR-03	The system shall store sensor readings in a database.
FR-04	The system shall display the latest soil-moisture, temperature, and humidity readings.
FR-05	The system shall request an irrigation prediction from the configured model service.
FR-06	The system shall record the predicted class, model version, and prediction time.
FR-07	The user shall be able to review historical readings and predictions.
FR-08	The user shall be able to view device connectivity and pump status.
FR-09	The system shall support manual pump commands when a device is connected and safety conditions permit.
FR-10	The system shall reject invalid or stale readings and show an understandable error.

FR-11	The system shall keep automatic pump control disabled until separately enabled after testing.

3.2 Non-Functional Requirements

ID	Requirement
NFR-01	The interface should be understandable to a novice user.
NFR-02	Passwords, if authentication is implemented, must be hashed using a suitable password-hashing method.
NFR-03	Sensor values must be validated before storage or control decisions.
NFR-04	The application should separate UI, business logic, database, and device communication.
NFR-05	The system should log errors and important device/control events.
NFR-06	The system should tolerate temporary network failure without leaving the pump running indefinitely.
NFR-07	AI results must include model version and be evaluated on data not used for training.
NFR-08	The system should be maintainable and testable.

3.3 Assumptions and Constraints

- A working Wi-Fi network is available during demonstration.
- Sensors are calibrated for the soil and setup being tested.
- The model training data contains meaningful labels from reliable agricultural guidance or validated observations.
- Public datasets may not match local soil, crop, sensor, or weather conditions.
- The first release is a student prototype, not a certified agricultural control product.

- Mains-voltage pump wiring must be handled by a qualified person using properly rated, enclosed components.

Chapter 4

Feasibility and Risk Analysis

4.1 Feasibility

Type	Assessment
Technical	ESP32, MQTT, Java, MySQL, and common ML libraries can support the proposed prototype. Integration requires staged testing.
Economic	Open-source development tools can reduce software cost. Hardware cost depends on sensor quality, pump, enclosure, power supply, and local prices.
Operational	A simple dashboard can help a grower interpret measurements, but recommendations must be explained clearly.
Schedule	An eight-week schedule is plausible for a limited prototype if scope is controlled.
Privacy and security	Device credentials, account data, and farm information must be protected; public deployment requires stronger security.

4.2 Risk Register

Risk	Impact	Mitigation

Unreliable sensor readings	Incorrect recommendations	Calibrate sensors, filter impossible values, and record sensor quality.
Insufficient labeled data	Weak model performance	Begin with a suitable dataset; collect field observations; clearly disclose limitations.
Data leakage in evaluation	Overstated model score	Use time-aware or group-aware splits where appropriate; keep a final test set untouched.
Wi-Fi or MQTT outage	Missing readings or stale commands	Use timestamps, connection status, command expiry, and local fail-safe behavior.
Pump remains on	Crop/property damage or electrical risk	Add maximum run time, manual emergency stop, independent controller limits, and qualified wiring.
Project scope grows	Missed deadline	Prioritize core features and defer advanced prediction features.

Chapter 5

Software Process Model

The project will use the **Iterative Model**. The application will be developed in small working increments. Each iteration includes planning, design, implementation, testing, and review. This approach is appropriate because the sensor integration and AI model may need adjustment after early results.

5.1 Iterations

1. **Iteration 1: Requirements and design** — define the problem, requirements, database, and diagrams.
2. **Iteration 2: IoT data capture** — read sensors and transmit timestamped data.
3. **Iteration 3: Java application** — receive/store data and display a basic dashboard.
4. **Iteration 4: AI model** — prepare data, train candidate models, evaluate them, and integrate inference.
5. **Iteration 5: Control, testing, and finalization** — add safe manual control, test end-to-end behavior, and prepare the report.

Chapter 6

Project Schedule, Milestones, and Deliverables

6.1 Eight-Week Schedule

Week	Activities	Milestone	Deliverable
1	Problem analysis, objectives, stakeholder and requirement gathering	M1: Scope approved	Initial proposal and requirements list
2	Architecture, process model, risks, and database planning	M2: Requirements baseline	Requirements specification and architecture
3	DFD, use case, activity, ERD, class, and object diagrams	M3: Design reviewed	Diagram set and schema draft
4	ESP32 sensor reading and message transport	M4: Sensor data received	Firmware prototype and sample readings
5	Java backend, database persistence, and interface skeleton	M5: Data stored and displayed	Backend/database prototype
6	Dataset preparation, model training, evaluation, and inference integration	M6: AI evaluated	Model artifact, evaluation report, baseline comparison

7	End-to-end integration, safety checks, and functional testing	M7: Integrated prototype tested	Test cases, results, bug list
8	Final documentation, demonstration, and submission packaging	M8: Project submitted	Final report, source code, demo, presentation

6.2 Gantt Chart

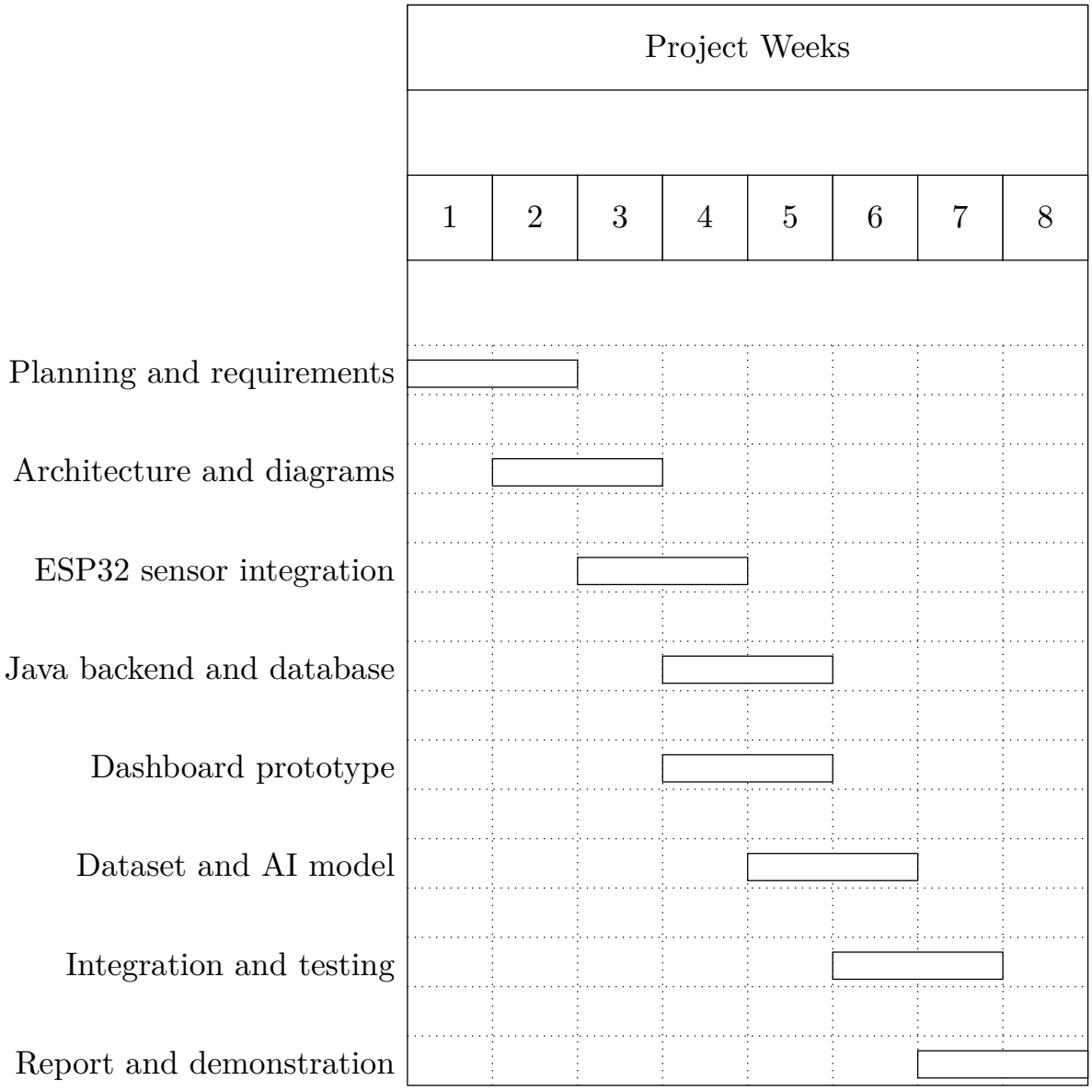


Figure 6.1: Planned eight-week project schedule.

Chapter 7

System Architecture and AI Plan

7.1 Architecture

The ESP32 reads sensors and sends timestamped measurements through MQTT. A Java Spring Boot backend validates the data and stores it in MySQL. The backend requests a prediction from a trained model, either through a Python inference service or a compatible Java inference runtime. The browser dashboard shows readings, model predictions, and device status.

7.2 AI Problem Definition

The initial AI task is supervised binary classification:

- **Input features:** calibrated soil-moisture measurement, temperature, humidity, and optionally rainfall, crop type, soil type, and growth stage when reliable data is available.
- **Target label:** whether irrigation is needed according to a documented agronomic rule or reliable field observation.
- **Candidate models:** Decision Tree as an interpretable baseline; Random Forest as a comparison.
- **Baseline:** a documented rule-based irrigation policy. The ML model should be compared with it.
- **Evaluation:** precision, recall for the irrigation-needed class, F1-score, confusion matrix, and accuracy. Report the dataset source and limitations.

7.3 Training Workflow

1. Obtain a dataset with suitable inputs and reliable irrigation labels, or collect labeled measurements under a documented protocol.

2. Inspect units, sensor ranges, missing values, duplicate rows, and class balance.
3. Split data into training, validation, and final test sets. For time-series sensor data, prefer chronological or group-aware evaluation to reduce leakage.
4. Fit preprocessing only on training data.
5. Train candidate models and tune them using training/validation data.
6. Evaluate once on the held-out test set and compare with the rule-based baseline.
7. Save the model, feature order, preprocessing configuration, and model version together.
8. Integrate inference into Java and monitor errors and out-of-range inputs.

7.4 AI Limitations

A model trained on synthetic or threshold-generated labels may merely reproduce the rule used to create those labels. Such data can demonstrate the software pipeline but cannot prove agricultural usefulness. A public dataset may not match the local crop, soil, sensor, or climate. The system must describe the output as a recommendation until validated in the intended setting. It must not claim water savings or improved crop yield without a controlled comparison.

Chapter 8

System Diagrams

8.1 Data Flow Diagram (Level 1)

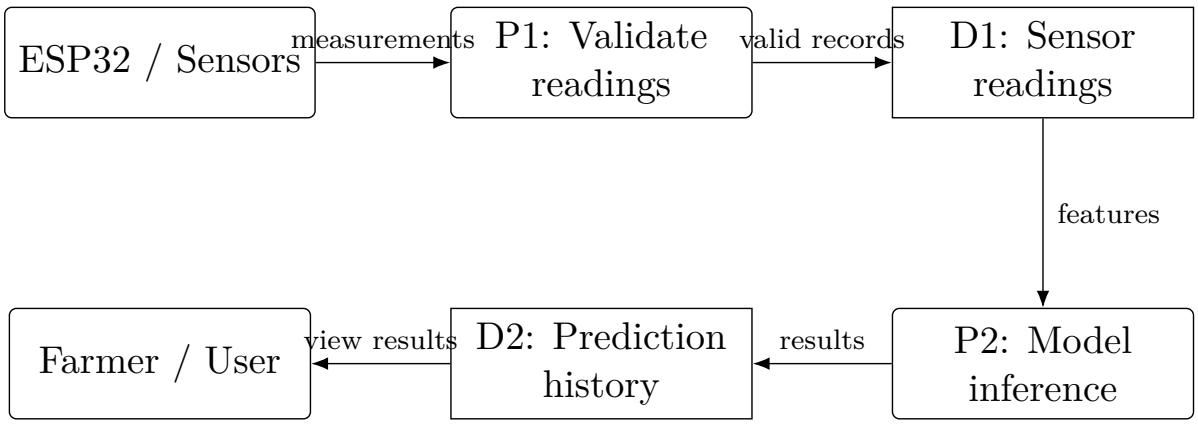


Figure 8.1: Level 1 data flow: measurements, validation, inference and review.

8.2 Use Case Diagram

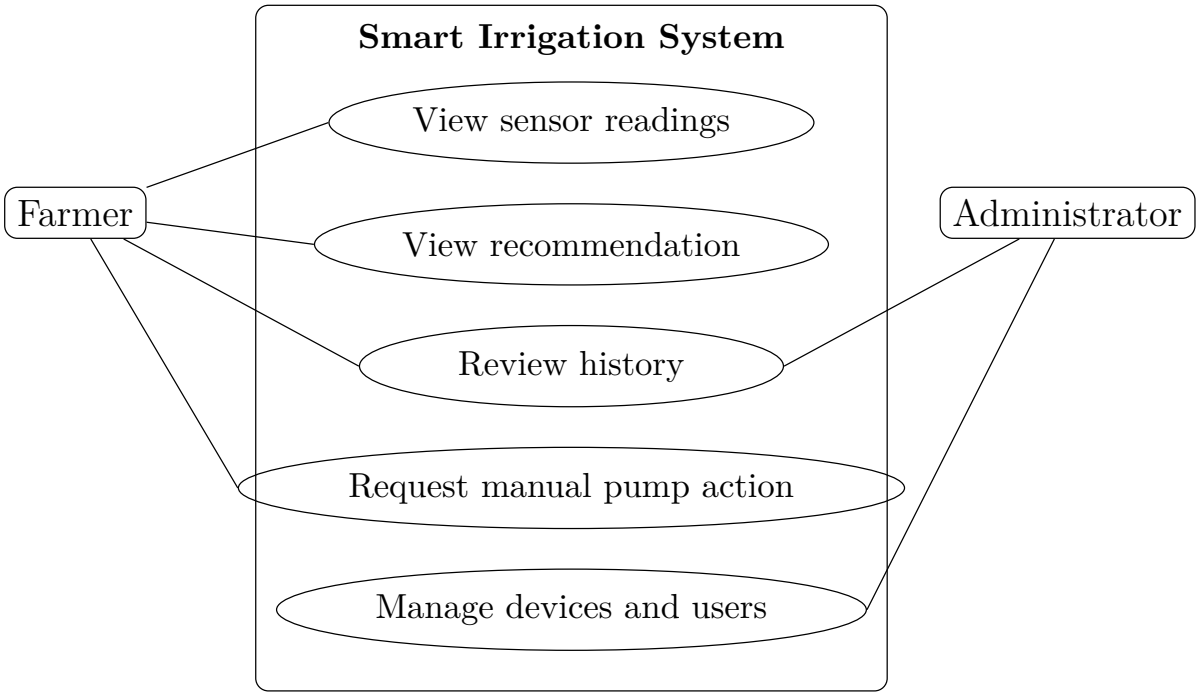


Figure 8.2: Use cases within the system boundary; lines denote actor associations.

8.3 Activity Diagram

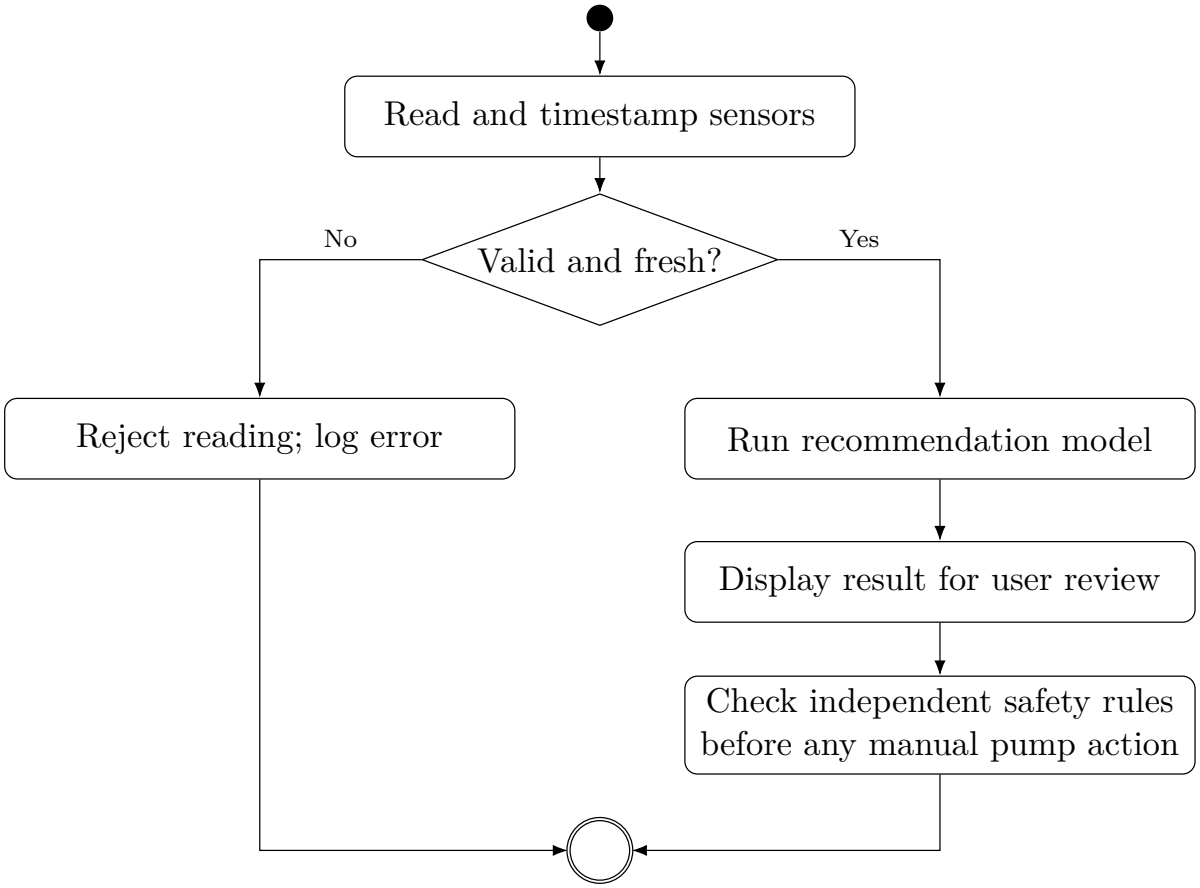


Figure 8.3: Activity workflow with invalid-reading rejection and independent safety checks.

8.4 Entity Relationship Diagram

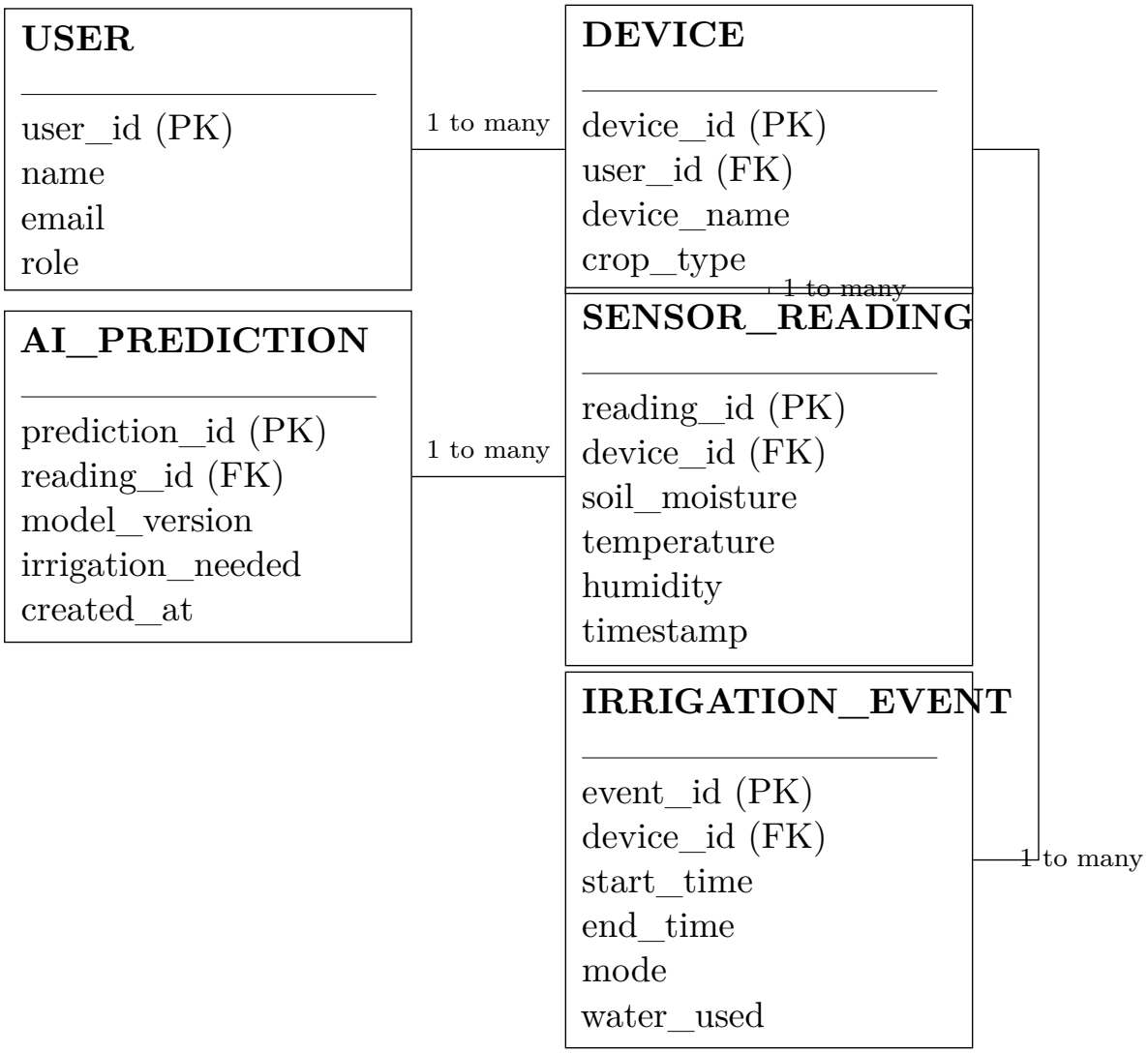


Figure 8.4: Logical entity relationships; PK = primary key and FK = foreign key.

8.5 Class Diagram

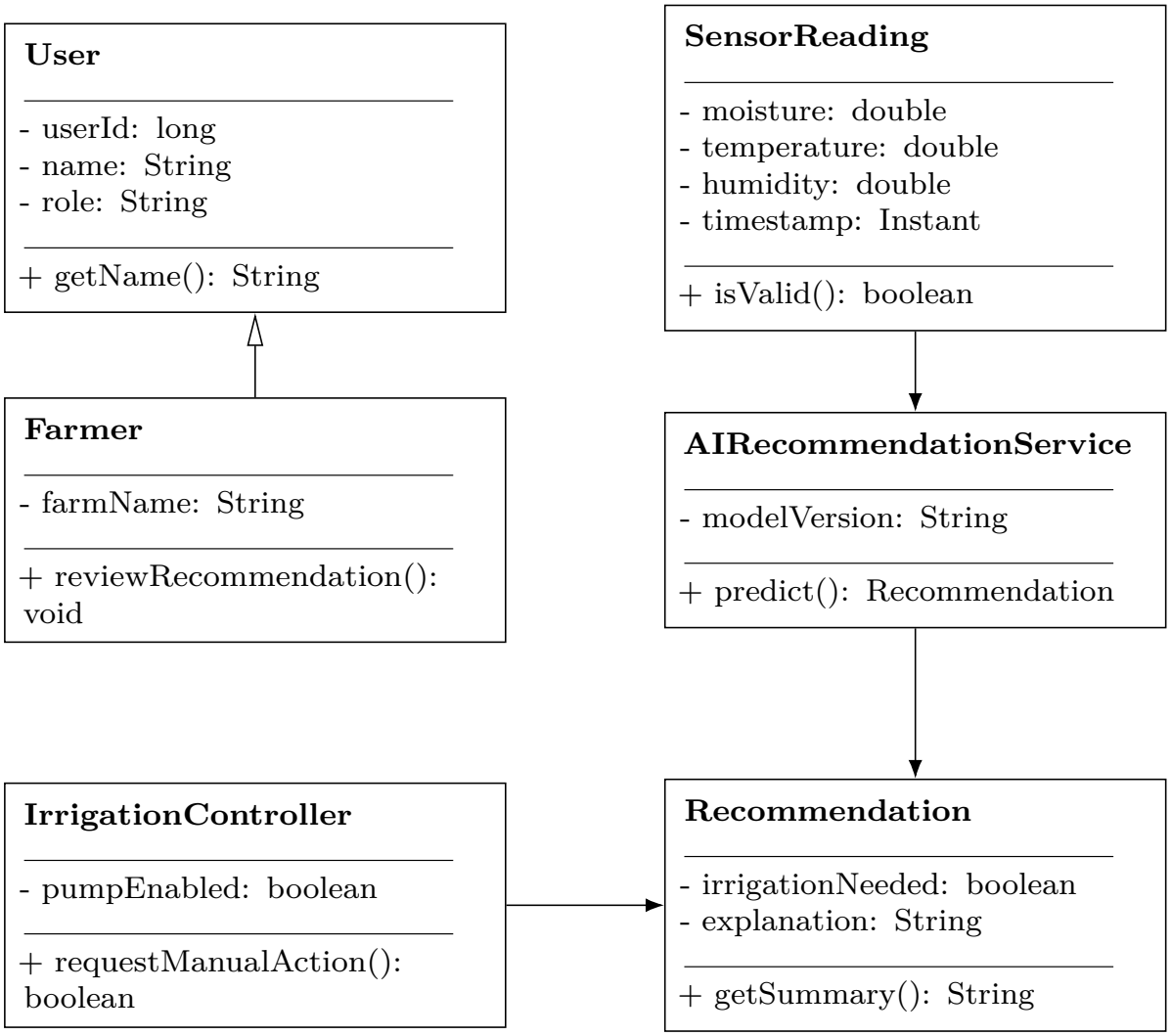


Figure 8.5: Simplified UML class relationships; hollow triangle indicates inheritance.

8.6 Object Diagram

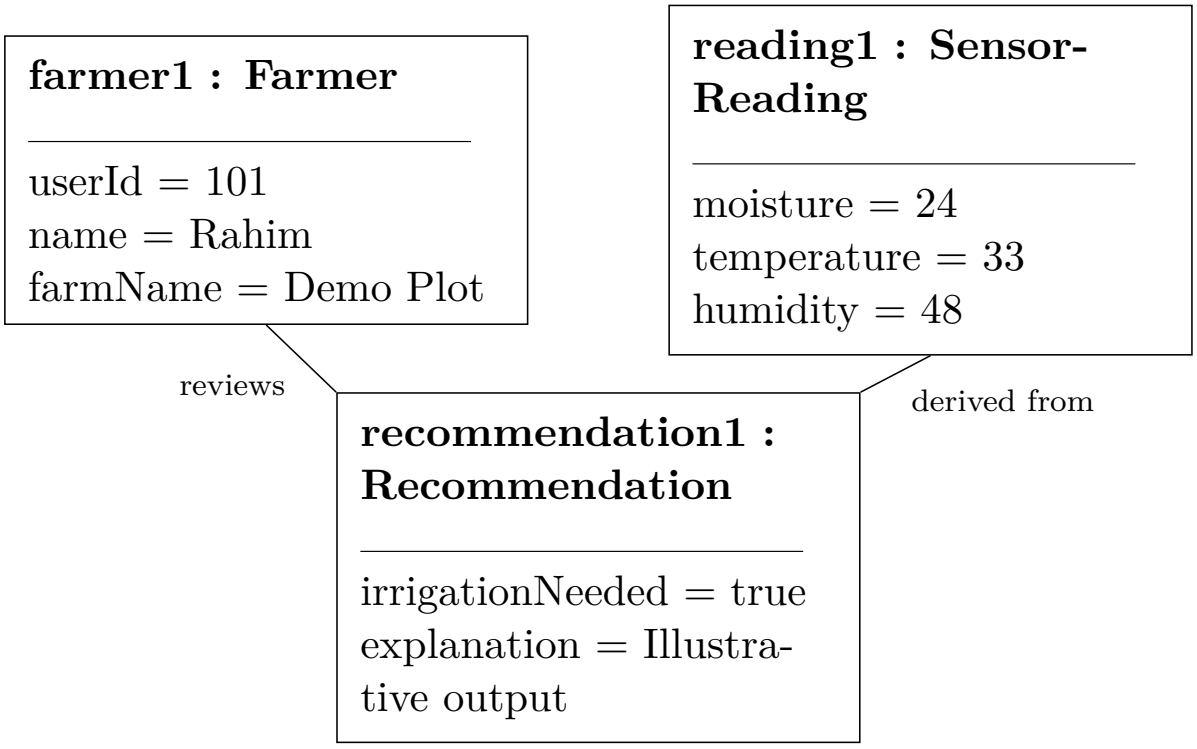


Figure 8.6: Illustrative object instances; sample values are not field-validated.

Chapter 9

Implementation Plan and Interface Prototype

9.1 Suggested Software Structure

- **Device layer:** ESP32 firmware reads sensors and publishes MQTT messages.
- **Backend layer:** Java Spring Boot validates readings, exposes REST endpoints, and coordinates inference.
- **Data layer:** MySQL stores devices, readings, predictions, and irrigation events.
- **AI layer:** a trained Decision Tree or Random Forest is served through a Python API or a compatible Java inference runtime.
- **UI layer:** a browser dashboard shows readings, recommendations, and history.

9.2 Java Class Responsibilities

The initial class design includes at least five classes: `User`, `Farmer`, `SensorReading`, `Recommendation`, `AIRecommendationService`, and `IrrigationController`. The supplied desktop prototype uses these classes to demonstrate object-oriented structure. It is a user-interface prototype, not a complete cloud-connected application.

9.3 Prototype Features

- Input soil moisture, temperature, and humidity values.
- Validate that values are within configured demo ranges.
- Generate an explicitly labeled demonstration recommendation.

- Display a recommendation summary in the interface.
- Keep pump action disabled in the supplied prototype.

9.4 Important AI Implementation Note

The included desktop interface uses a clearly marked demonstration recommendation service so the interface can run before a trained model is available. Its output must not be described as a trained AI prediction. Replace the demonstration implementation with an evaluated model integration before claiming live AI functionality.

Chapter 10

Testing and Evaluation

10.1 Test Plan

Test ID	Test	Expected result
T-01	Enter valid sensor values	Values accepted and displayed.
T-02	Enter moisture outside configured range	Input rejected with a clear message.
T-03	Submit a prediction request	A demo recommendation is displayed; later, the trained model response is displayed.
T-04	Send missing sensor fields	Backend rejects or marks the record invalid.
T-05	Simulate a stale reading	No pump command is issued.
T-06	Simulate MQTT/network failure	UI reports device offline; no indefinite pump operation.
T-07	Evaluate the trained model on held-out data	Metrics and confusion matrix are saved in the report.
T-08	Compare AI with baseline rule	Report both results using the same test protocol.
T-09	Attempt pump command with safety condition failed	Command is blocked.

T-10	Review historical data	Records are shown in chronological order.

10.2 AI Evaluation Metrics

Report accuracy, precision, recall, F1-score, and the confusion matrix. Prioritize recall for the irrigation-needed class while also reporting false-positive recommendations, because unnecessary irrigation also has a cost. Use a final test set not used for model selection. Document the dataset, label-generation method, and known limitations.

Chapter 11

Expected Outcomes

1. A documented project proposal and requirements specification.
2. Six system diagrams.
3. A working sensor-data capture prototype.
4. A Java application prototype with a class-based interface.
5. A database design for readings and predictions.
6. An evaluated machine-learning model or a clearly documented model-training pipeline.
7. Test results, risk analysis, and final presentation.

Chapter 12

Future Enhancements

Potential future work includes crop-specific model evaluation, rainfall forecast integration, water-flow measurement, abnormal sensor detection, mobile notifications, multiple field support, and safe automated irrigation after field validation.

Chapter 13

Conclusion

The proposed system combines IoT sensing, machine learning, and Java software engineering to support irrigation decisions. The initial version focuses on sensor monitoring, reliable data storage, and transparent recommendations. Its success will be assessed through functional tests and an honest evaluation of the AI model against a documented baseline. Automatic pump control will remain protected by independent safety limits and will not rely on an unvalidated model alone.

Chapter 14

References

1. Scikit-learn documentation, https://scikit-learn.org/stable/.
2. Spring Boot documentation, https://spring.io/projects/spring-boot.
3. Eclipse Paho MQTT client documentation, https://eclipse.dev/paho/.
4. MySQL documentation, https://dev.mysql.com/doc/.
5. FAO AQUASTAT, https://www.fao.org/aquastat/en/.
6. UCI Machine Learning Repository, https://archive.ics.uci.edu/.

Appendix A

Submission Notes

Replace the student details on the cover page. Confirm that the course instructor accepts a Java desktop prototype for the interface requirement. If the instructor requires a web interface, implement the same classes behind a Spring Boot web application and use HTML/CSS/JavaScript for the browser UI. Add actual implementation screenshots, dataset provenance, measured model results, and the final test evidence before submission.